UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática

TFG del Grado en Ingeniería Informática

Addon Blender 1.0

Presentado por María Molina Goyena
en Universidad de Burgos — 22 de marzo de 2026
Tutores: Bruno Rodrígez García y Raúl Marticorena Sánchez

Índice general

Índice general **i**

Índice de figuras **iii**

Índice de tablas **iv**

.1. Introducción 1
.2. Planificación temporal 1
.3. Estudio de viabilidad 5

Apéndice A Especificación de Requisitos **7**

A.1. Casos de uso 7
A.2. Objetivos generales 14
A.3. Diagramas del sistema 14
A.4. Catálogo de requisitos 16
A.5. Especificación de requisitos 18

Apéndice B Especificación de diseño **21**

B.1. Introducción 21
B.2. Diseño de datos 22
B.3. Diseño arquitectónico 24
B.4. Diseño procedimental 27
B.5. Decisiones de diseño relevantes 29

Apéndice C Documentación técnica de programación **33**

C.1. Introducción 33
C.2. Estructura de directorios 33
C.3. Manual del programador 34

C.4. Compilación, instalación y ejecución del proyecto 35
C.5. Pruebas del sistema 36

Apéndice D Documentación de usuario 39
D.1. Introducción 39
D.2. Requisitos de usuario 39
D.3. Instalación 40
D.4. Manual del usuario 41

Apéndice E Anexo de sostenibilización curricular 43
E.1. Introducción 43
E.2. Sostenibilidad tecnológica 43
E.3. Sostenibilidad educativa 44
E.4. Sostenibilidad social 44
E.5. Conclusión 44

Índice de figuras

A.1. Diagrama UML de casos de uso del sistema. 15
A.2. Diagrama de flujo de captura, registro y análisis de eventos. . . 16

B.1. Estructura lógica de los datos registrados por el sistema. 23
B.2. Arquitectura vertical del sistema. 25
B.3. Diagrama de componentes del sistema. 26
B.4. Flujo procedimental del sistema. 28
B.5. Diagrama de secuencia del sistema. 29
B.6. Arquitectura general del sistema basada en dos addons independientes. 31

Índice de tablas

A.1. CU-1: Registro de creación de objetos. 8
A.2. CU-2: Registro de modificaciones de geometría. 9
A.3. CU-3: Registro de cambios de modo. 10
A.4. CU-4: Registro de eliminación de objetos. 11
A.5. CU-5: Visualización de métricas. 12
A.6. CU-6: Exportación de datos. 13
A.7. Especificación completa de requisitos, relacionando RF, RNF, RIU y RLE con los casos de uso. 19

.1. Introducción

El presente plan de proyecto tiene como objetivo establecer las bases organizativas y metodológicas para el desarrollo del addon *Análisis 3D CSV* en Blender. Este documento define la planificación del desarrollo, los recursos necesarios, los criterios de viabilidad y las estrategias de seguimiento del proyecto, con el fin de garantizar que se cumplan los objetivos establecidos en el TFG.

La definición de requisitos se realizó previamente al inicio del desarrollo. A partir de ese punto, el proyecto se ha desarrollado siguiendo un enfoque iterativo basado en ciclos de trabajo de dos semanas (sprints), permitiendo evolucionar el producto de forma incremental mediante distintas versiones funcionales del addon.

El desarrollo se ha basado en principios de metodologías ágiles, inspiradas en Scrum, mediante iteraciones cortas y evolución incremental del producto.

El repositorio GitHub ha actuado como herramienta central de gestión del proyecto, permitiendo el control de versiones, la trazabilidad del desarrollo y el seguimiento de la evolución del sistema mediante commits, incidencias (*issues*) y versiones etiquetadas.

.2. Planificación temporal

La planificación del proyecto se ha estructurado en sprints quincenales, alineados con la evolución real del desarrollo. El seguimiento de cada sprint se ha realizado mediante el repositorio de GitHub, utilizando *commits*, *issues* y versiones etiquetadas (*tags*).

- **Fase previa (Noviembre)**: Estudio de viabilidad del proyecto. Se realizaron pruebas iniciales para evaluar la posibilidad de desarrollar un sistema de recopilación y análisis de datos en Blender, incluyendo el análisis de su API, la viabilidad técnica de la captura de eventos y la posibilidad de generar métricas a partir de los datos registrados.
-
- **Sprint 0 (Enero)**: Definición de requisitos, análisis del problema y estudio de la API de Blender.
- **Sprint 1 (Febrero - semanas 1-2)**: Desarrollo del primer addon funcional. Implementación inicial de la captura de eventos y expor-

tación a CSV. *Resultado: versión inicial funcional del sistema (tag: v0.1)*

- **Sprint 2 (Febrero - semanas 3-4)**: Mejora del primer addon, refinando la captura de datos y estructuración de la información.
- **Sprint 3 (Marzo - semanas 1-2)**: Desarrollo del segundo addon, ampliando funcionalidades y mejorando la lógica de procesamiento de datos. *Resultado: ampliación del sistema (tag: v0.2)*
- **Sprint 4 (Marzo - semanas 3-4)**: Incorporación de nuevas métricas y mejora del análisis de datos.
- **Sprint 5 (Abril - semanas 1-2)**: Refactorización del sistema y mejora significativa de funcionalidades clave. *Commit destacado: - efactorización del sistema de logging y detección de eventos (v0.4)"*

  En este sprint se introducen mejoras relevantes:

  - Soporte completo para Edit Mode mediante *bmesh*.
  - Mejora en la detección de operadores (Shift+D, Alt+D, Ctrl+V, Merge).
  - Detección robusta de coordenadas UV.
  - Optimización del sistema de logging, registrando únicamente cambios reales.
  - Implementación de controles de ejecución (Start/Stop).
  - Incorporación de un panel de depuración.
  - Mejora en la generación y estructura de los archivos CSV.

  Resultado: versión avanzada y estable del sistema (tag: v0.4)
- **Sprint 6 (Abril - semanas 3-4)**: Mejora de la usabilidad y ajustes funcionales adicionales.
- **Sprint 7 (Mayo - semanas 1-2)**: Pruebas funcionales y corrección de errores.
- **Sprint 8 (Mayo - semanas 3-4)**: Documentación final y preparación de la entrega. *Resultado: versión final del sistema (tag: v1.0)*

La información reflejada en esta planificación se corresponde con la evolución real del repositorio del proyecto en GitHub, donde pueden consultarse los commits y versiones asociadas a cada iteración.

Este enfoque permite reflejar un desarrollo incremental del sistema, donde cada sprint aporta mejoras o nuevas funcionalidades, apoyándose en el control de versiones para garantizar la trazabilidad del proyecto.

Resumen de sprints

A continuación, se presenta un resumen de los sprints realizados durante el desarrollo del proyecto, incluyendo las principales tareas llevadas a cabo y los resultados obtenidos en cada iteración.

Sprint	**Periodo**	**Funcionalidades principales**	**Resultado**
Sprint 1	Febrero (1-2)	Estructura inicial del addon, captura de eventos y exportación a CSV	Versión inicial (v0.1)
Sprint 2	Febrero (3-4)	Mejora del sistema de captura de datos y formato CSV	Sistema más robusto
Sprint 3	Marzo (1-2)	Desarrollo del segundo addon y ampliación del procesamiento de datos	Versión ampliada (v0.2)
Sprint 4	Marzo (3-4)	Implementación de métricas y análisis de datos	Sistema analítico inicial
Sprint 5	Abril (1-2)	Refactorización del sistema, mejoras en logging y detección de eventos (Edit Mode, operadores, UV)	Versión avanzada (v0.4)
Sprint 6	Abril (3-4)	Mejora de usabilidad, integración y optimización general	Sistema optimizado
Sprint 7	Mayo (1-2)	Pruebas funcionales, validación y corrección de errores	Sistema validado
Sprint 8	Mayo (3-4)	Documentación final y preparación de la entrega	Versión final (v1.0)

La información reflejada en esta tabla se corresponde con la evolución del repositorio del proyecto en GitHub.

Viabilidad económica

Aunque el proyecto se desarrolla en un entorno académico, es posible estimar su coste aproximado considerando un escenario profesional. Para ello, se tienen en cuenta los costes de software, hardware y desarrollo.

Costes de software

El proyecto utiliza principalmente herramientas de software libre, por lo que no implica costes directos de licencia:

- Blender (GPL): 0 €
- Python (PSF License): 0 €
- GitHub (plan gratuito): 0 €

No obstante, el uso de un sistema operativo propietario como Windows puede implicar un coste aproximado de licencia de 150 €.

Costes de hardware

Se ha utilizado un equipo personal con un valor estimado de 1200 €. Considerando una amortización en 4 años (48 meses) y una duración del proyecto de aproximadamente 4 meses, el coste imputable sería:

$$\text{Coste hardware} = \frac{1200}{48} \times 4 = 100\,£$$

Coste de desarrollo

Para estimar el coste de desarrollo, se considera un perfil de desarrollador junior con un salario aproximado de 1200 € mensuales durante 4 meses:

$$\text{Coste desarrollo} = 1200 \times 4 = 4800\,£$$

Coste total estimado

$$\text{Coste total} = 150 + 100 + 4800 = 5050\,£$$

Por tanto, aunque el coste real del proyecto es reducido al tratarse de un entorno académico, su equivalente en un contexto profesional sería de aproximadamente 5050 €.

Este análisis demuestra que el sistema es económicamente viable, especialmente al basarse en herramientas de software libre que reducen significativamente los costes de desarrollo.

.3. Estudio de viabilidad

Viabilidad legal

Dado que Blender se distribuye bajo licencia GPL, los addons desarrollados para esta plataforma deben ser compatibles con dicha licencia. Por ello, el producto final se distribuye bajo licencia GPL (GNU General Public License), garantizando la libertad de uso, modificación y distribución del software.

En caso de utilizar bibliotecas de terceros, estas deberán ser compatibles con dicha licencia (por ejemplo, MIT o BSD).

Asimismo, el sistema incluye mecanismos de consentimiento del usuario para la recopilación de datos, cumpliendo con la normativa vigente en materia de protección de datos.

Análisis de riesgos

Como principal riesgo del proyecto se identifica la complejidad de la API de Blender, especialmente en la detección de eventos en distintos modos de trabajo como *Object Mode* y *Edit Mode*.

Este riesgo se ha mitigado mediante un desarrollo iterativo, pruebas continuas y refactorizaciones progresivas del sistema, como la realizada en la versión v0.4.

Este planteamiento permite alinear la planificación teórica con la implementación real del sistema, garantizando coherencia entre la documentación y el desarrollo del proyecto.

Apéndice A

Especificación de Requisitos

En este apartado se presenta la especificación de requisitos del sistema desarrollado, compuesto por dos addons independientes para Blender: un addon de recopilación de datos y un addon de análisis de datos y métricas.

En primer lugar, se describen los casos de uso más representativos del sistema, ya que constituyen la base para la obtención de los requisitos funcionales. Posteriormente, se recogen los objetivos generales, los diagramas que modelan el comportamiento y la arquitectura del sistema, y finalmente el catálogo completo de requisitos funcionales, no funcionales, de interfaz y legales.

A.1. Casos de uso

A continuación, se describen los casos de uso principales del sistema. Las Tablas A.1 a A.6 recogen de forma estructurada la información esencial de cada caso de uso, incluyendo su descripción, precondiciones, flujo de acciones, postcondiciones y posibles excepciones.

CU-1	Registrar creación de objeto en la escena
Versión	1.0
Autor	María
Requisitos asociados	RF-01, RF-10
Descripción	Captura automática de la creación de un objeto por parte del usuario, incluyendo propiedades geométricas y posición espacial.
Precondición	El addon de recopilación de datos está activado y la sesión de Blender está abierta.
Acciones	1. El usuario crea un nuevo objeto en la escena. 2. El módulo de captura detecta la acción. 3. Se extraen las propiedades relevantes del objeto. 4. Los datos se almacenan en el archivo CSV correspondiente.
Postcondición	El evento queda registrado y disponible para su análisis posterior.
Excepciones	Error en la captura debido a conflicto con otro addon o fallo interno de Blender.
Importancia	Alta

Tabla A.1: CU-1: Registro de creación de objetos.

CU-2	Registrar modificaciones de geometría de un objeto
Versión	1.0
Autor	María
Requisitos asociados	RF-02, RF-10
Descripción	Detecta cambios en la geometría de los objetos, incluyendo vértices, aristas y caras, y registra las modificaciones realizadas.
Precondición	El addon de recopilación de datos está activo y el objeto existe en la escena.
Acciones	1. El usuario realiza modificaciones en el objeto, por ejemplo edición de malla, extrusión o combinación. 2. El sistema detecta los cambios geométricos mediante handlers de Blender. 3. Se registran las propiedades previas y actuales del objeto. 4. Los datos se almacenan para análisis temporal y estructural posterior.
Postcondición	La modificación queda registrada con información suficiente para reconstruir los cambios relevantes.
Excepciones	Error en la captura si la malla es demasiado compleja o existe conflicto con otro addon.
Importancia	Alta

Tabla A.2: CU-2: Registro de modificaciones de geometría.

CU-3	Registrar cambio de modo de edición
Versión	1.0
Autor	María
Requisitos asociados	RF-03, RF-10
Descripción	Detecta y registra los cambios de modo de trabajo en Blender, especialmente entre *Object Mode* y *Edit Mode*.
Precondición	El addon de recopilación de datos está activo.
Acciones	1. El usuario cambia el modo de edición de un objeto. 2. El sistema detecta el cambio mediante handlers de Blender. 3. Se registra el modo anterior, el modo actual y el objeto afectado.
Postcondición	El cambio de modo queda registrado con su marca temporal correspondiente.
Excepciones	Fallo en la captura si la escena está bloqueada o el objeto ya no existe.
Importancia	Media

Tabla A.3: CU-3: Registro de cambios de modo.

CU-4	Registrar eliminación de objeto
Versión	1.0
Autor	María
Requisitos asociados	RF-04, RF-10
Descripción	Registra la eliminación de un objeto de la escena, almacenando sus propiedades relevantes antes de su desaparición.
Precondición	El addon de recopilación de datos está activo y el objeto existe en la escena.
Acciones	1. El usuario elimina un objeto. 2. El addon detecta la eliminación mediante handlers. 3. Se registran propiedades clave del objeto eliminado y la marca temporal de la acción.
Postcondición	La eliminación queda registrada para su análisis histórico posterior.
Excepciones	Fallo si la escena está corrupta o existe conflicto con otro addon.
Importancia	Alta

Tabla A.4: CU-4: Registro de eliminación de objetos.

CU-5	Visualizar métricas de modelado
Versión	1.0
Autor	María
Requisitos asociados	RF-05, RF-07
Descripción	Permite consultar métricas temporales, espaciales y estadísticas obtenidas a partir de la sesión de modelado registrada.
Precondición	El addon de análisis de datos tiene acceso a los registros de la sesión.
Acciones	1. El usuario abre el panel de métricas. 2. El addon calcula las métricas a partir de los datos registrados. 3. Se muestran resúmenes estadísticos y resultados filtrables por objeto, evento o rango temporal.
Postcondición	Las métricas quedan visibles en el panel del usuario.
Excepciones	Datos incompletos o corruptos; error en el cálculo de métricas.
Importancia	Alta

Tabla A.5: CU-5: Visualización de métricas.

CU-6	Exportar datos a archivo externo
Versión	1.0
Autor	María
Requisitos asociados	RF-06
Descripción	Permite exportar los datos registrados para su consulta o tratamiento externo.
Precondición	Existen datos previamente registrados y el addon de análisis está activo.
Acciones	1. El usuario selecciona la opción de exportación. 2. El addon prepara los registros almacenados. 3. El archivo se guarda en la ubicación indicada por el usuario.
Postcondición	Los datos quedan disponibles para análisis externo.
Excepciones	Error de escritura o permisos insuficientes en la ruta de destino.
Importancia	Media

Tabla A.6: CU-6: Exportación de datos.

A.2. Objetivos generales

A partir de los casos de uso descritos en las Tablas A.1 a A.6, se establecen los objetivos generales que debe satisfacer el sistema:

- Registrar de manera precisa los eventos relevantes producidos durante el modelado 3D.
- Permitir el análisis estadístico y la visualización de los datos recopilados.
- Garantizar la persistencia, integridad y trazabilidad de los datos entre sesiones.
- Proporcionar una interfaz clara y segura para la interacción con ambos addons.
- Cumplir con la normativa legal y ética relativa a la protección de datos y al uso de software libre.

A.3. Diagramas del sistema

Con el fin de complementar la descripción textual del sistema, se incluyen a continuación varios diagramas que representan sus funcionalidades, flujo de trabajo, secuencia de interacción y arquitectura general.

En primer lugar, la Figura A.1 muestra el diagrama UML de casos de uso. En él se representa al usuario como actor principal y se identifican las funcionalidades más importantes del sistema. Desde el punto de vista funcional, los casos de uso *Crear objeto*, *Modificar geometría* y *Eliminar objeto* implican necesariamente la ejecución del caso de uso *Registrar evento*, En este contexto, las relaciones entre los casos de uso se corresponden conceptualmente con relaciones de tipo *«include»* y *«extend»*, aunque en el diagrama se han representado de forma simplificada para facilitar su lectura.

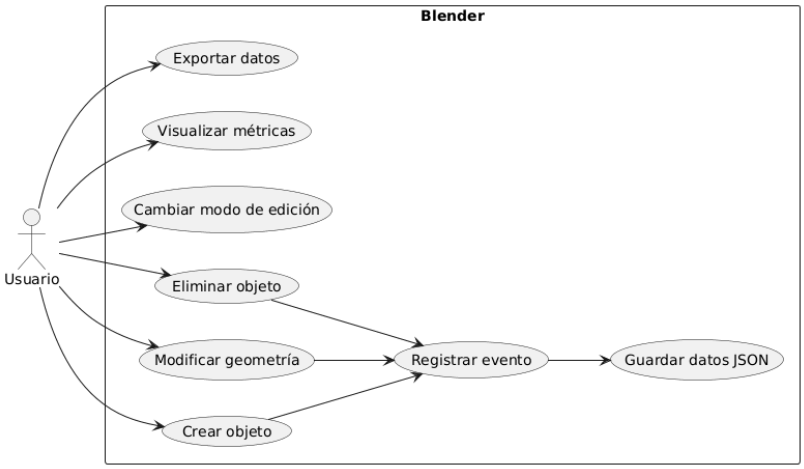

Figura A.1: Diagrama UML de casos de uso del sistema.

La Figura A.2 representa el flujo general de procesamiento del sistema, desde la interacción inicial del usuario con Blender hasta la visualización de métricas. Este diagrama permite observar cómo el sistema clasifica los eventos según su tipo, registra la información en CSV y la pone posteriormente a disposición del addon de análisis.

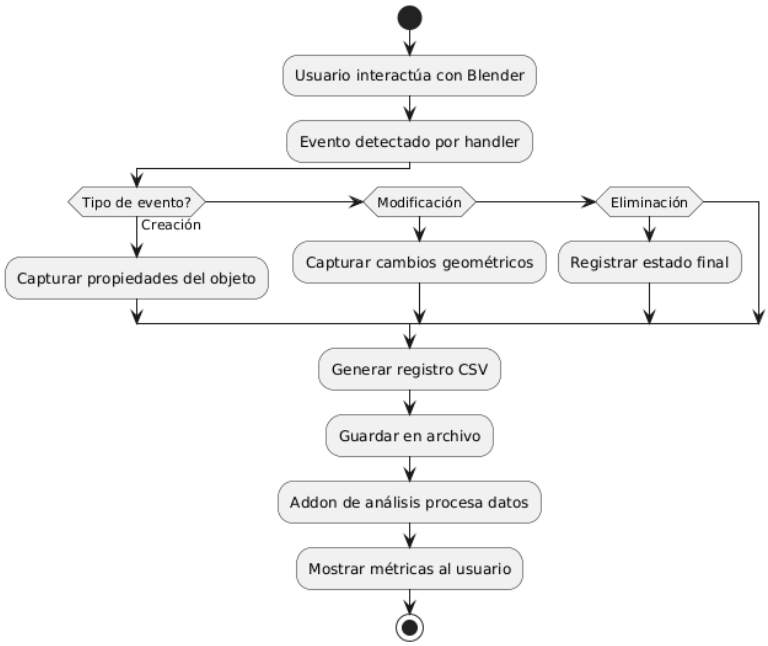

Figura A.2: Diagrama de flujo de captura, registro y análisis de eventos.

El comportamiento dinámico del sistema se describe con mayor detalle en el diagrama de secuencia incluido en el Apéndice C.

La Figura B.6 muestra una visión general del sistema, cuya arquitectura se desarrolla en detalle en el Apéndice C.

A.4. Catálogo de requisitos

A partir de los casos de uso y de los diagramas presentados en las Figuras A.1 a B.6, se define el catálogo de requisitos del sistema, clasificado en requisitos funcionales, no funcionales, de interfaz de usuario y legales.

Requisitos funcionales (RF)

- RF-01: Registrar la creación de objetos.
- RF-02: Registrar modificaciones de geometría.
- RF-03: Detectar cambios de modo de edición.

- RF-04: Registrar la eliminación de objetos.
- RF-05: Visualizar métricas de modelado.
- RF-06: Exportar datos a archivo externo.
- RF-07: Filtrar métricas por objeto, evento o tiempo.
- RF-08: Notificar errores en los datos o en su procesamiento.
- RF-09: Activar y desactivar la captura desde la interfaz de usuario.
- RF-10: Registrar marcas temporales precisas en cada evento.

Requisitos no funcionales (RNF)

- RNF-01: Eficiencia en la captura de eventos.
- RNF-02: Soporte de múltiples objetos y cambios complejos.
- RNF-03: Persistencia de datos ante cierres inesperados.
- RNF-04: Interfaz clara e intuitiva.
- RNF-05: Código mantenible y modular.

Requisitos de interfaz de usuario (RIU)

- RIU-01: Panel de control de captura.
- RIU-02: Panel de métricas con tablas o resúmenes visuales.
- RIU-03: Configuración de exportación de datos.
- RIU-04: Alertas de errores y estados del sistema.

Requisitos legales y éticos (RLE)

- RLE-01: Cumplimiento de la normativa de protección de datos.
- RLE-02: Consentimiento explícito del usuario para la recopilación de datos.
- RLE-03: Uso de software libre respetando las licencias aplicables.

A.5. Especificación de requisitos

Una vez definido el catálogo general, la Tabla A.7 recoge de forma consolidada la especificación completa de requisitos del sistema. En ella se relacionan los identificadores de requisito con su descripción, el addon responsable, su prioridad y el mecanismo de verificación asociado.

ID	Descripción	Addon	Prioridad	Verificación
RF-01	Registrar creación de objetos	Recopilación	Alta	CU-1
RF-02	Registrar modificaciones de geometría	Recopilación	Alta	CU-2
RF-03	Detectar cambios de modo de edición	Recopilación	Media	CU-3
RF-04	Registrar eliminación de objetos	Recopilación	Alta	CU-4
RF-05	Visualizar métricas de modelado	Análisis	Alta	CU-5
RF-06	Exportar datos a CSV/JSON	Análisis	Media	CU-6
RF-07	Filtrar métricas	Análisis	Media	CU-5
RF-08	Notificar errores en datos	Ambos	Media	Panel/Logs
RF-09	Activar/desactivar captura desde la UI	Recopilación	Alta	Panel de control
RF-10	Registrar marcas temporales precisas	Recopilación	Alta	Logs internos
RNF-01	Eficiencia en captura de eventos	Ambos	Alta	Pruebas de rendimiento
RNF-02	Soporte de múltiples objetos	Ambos	Alta	Pruebas de estrés
RNF-03	Persistencia ante cierres inesperados	Ambos	Alta	Pruebas de reinicio
RNF-04	Interfaz clara e intuitiva	Ambos	Media	Evaluación de usabilidad
RNF-05	Código mantenible y modular	Ambos	Media	Revisión de código
RIU-01	Panel de control de captura	Recopilación	Alta	UI test
RIU-02	Panel de métricas	Análisis	Alta	UI test
RIU-03	Configuración de exportación	Análisis	Media	UI test
RIU-04	Alertas de errores	Ambos	Media	UI test
RLE-01	Cumplimiento normativa protección de datos	Ambos	Alta	Documentación legal
RLE-02	Consentimiento explícito del usuario	Recopilación	Alta	Confirmación usuario
RLE-03	Uso software libre respetando licencias	Ambos	Media	Licencias verificadas

Tabla A.7: Especificación completa de requisitos, relacionando RF, RNF, RIU y RLE con los casos de uso.

Apéndice B

Especificación de diseño

B.1. Introducción

En este apartado se describe el diseño del sistema desarrollado, abordando su organización interna, la estructura de los datos gestionados y el comportamiento de los distintos módulos que componen los addons implementados para Blender.

El sistema está constituido por dos addons independientes pero complementarios: un addon de recopilación de datos, responsable de la detección y registro de eventos durante la interacción del usuario con el entorno de modelado, y un addon de análisis, encargado de procesar los datos generados y presentar métricas de forma visual. Ambos componentes se comunican mediante archivos intermedios en formato CSV, lo que permite mantener una arquitectura desacoplada, flexible y fácilmente extensible.

El diseño del sistema se ha realizado siguiendo principios clásicos de ingeniería del software, como la modularidad, la separación de responsabilidades, el bajo acoplamiento y la alta cohesión. Estos criterios permiten mejorar la mantenibilidad del sistema, facilitar su evolución futura y asegurar la coherencia entre los requisitos definidos, la implementación realizada y los resultados obtenidos.

Con el fin de estructurar la descripción del sistema de manera clara y completa, este apartado se organiza en tres niveles: diseño de datos, diseño arquitectónico y diseño procedimental.

B.2. Diseño de datos

El sistema utiliza archivos CSV como mecanismo principal de persistencia de datos. En estos archivos se almacenan los eventos capturados durante la interacción del usuario con Blender, permitiendo su posterior procesamiento por el addon de análisis.

La estructura de los datos se ha diseñado siguiendo un modelo semiestructurado orientado a eventos, en el que cada registro representa una acción significativa realizada por el usuario en el entorno de modelado. Este enfoque permite reconstruir la secuencia de acciones y analizar el comportamiento del usuario de forma temporal.

Cada fila del archivo corresponde a un evento detectado por el sistema. La información almacenada combina datos temporales, identificativos y geométricos, proporcionando una visión completa del estado del sistema en el momento de la interacción.

La Figura B.1 muestra la estructura lógica de los datos gestionados por el sistema.

Figura B.1: Estructura lógica de los datos registrados por el sistema.

De forma general, cada evento incluye los siguientes campos:

- **timestamp**: marca temporal exacta del evento.

- **tipo_evento**: tipo de acción registrada.

- **id_objeto**: identificador único del objeto afectado.

- **nombre_objeto**: nombre asignado al objeto en la escena.

- **modo**: modo de trabajo activo en el momento del evento.

- **propiedades**: conjunto de atributos geométricos o contextuales relevantes.

La elección del formato CSV responde a criterios de simplicidad, interoperabilidad y eficiencia, facilitando tanto el almacenamiento como la inspección manual y el procesamiento externo de los datos.

B.3. Diseño arquitectónico

La arquitectura del sistema se basa en un modelo modular orientado a la separación de responsabilidades. Este diseño sigue los principios de bajo acoplamiento y alta cohesión, permitiendo la evolución independiente de los distintos módulos y facilitando su mantenimiento.

La Figura B.2 presenta la arquitectura general del sistema en formato vertical, donde se muestra el flujo de datos desde la interacción del usuario hasta la visualización de métricas.

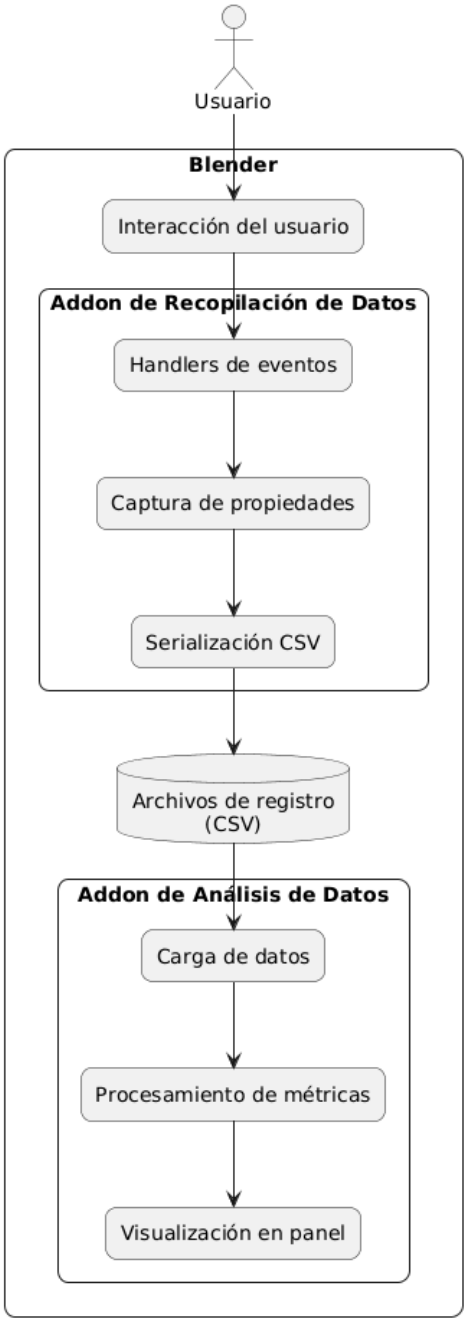

Figura B.2: Arquitectura vertical del sistema.

Asimismo, la Figura B.3 representa el sistema desde el punto de vista de componentes software, mostrando las dependencias y relaciones entre los distintos módulos.

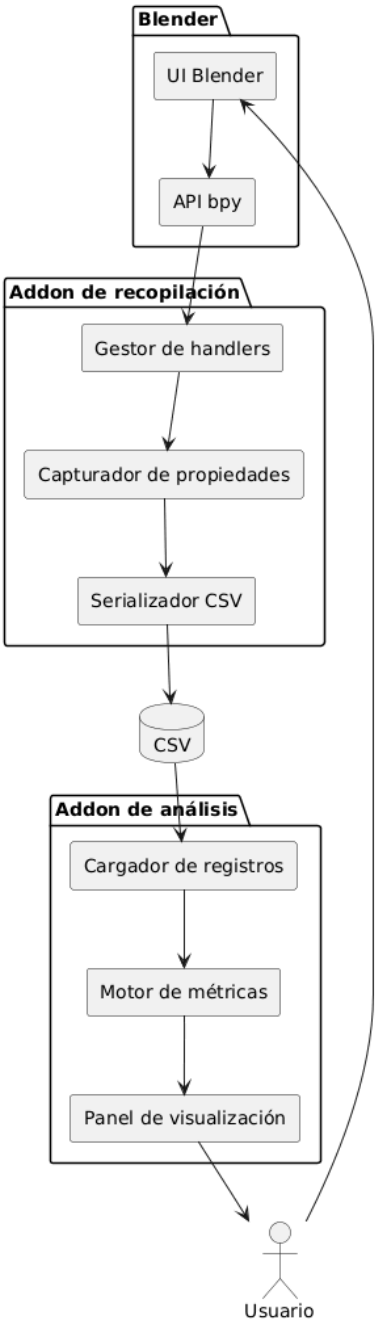

Figura B.3: Diagrama de componentes del sistema.

A nivel funcional, la arquitectura puede descomponerse en los siguientes bloques:

- Interacción del usuario en el entorno de Blender.
- Detección de eventos mediante handlers.
- Captura y estructuración de datos.
- Persistencia en archivos CSV.
- Carga y procesamiento de datos.
- Visualización de métricas en la interfaz.

Este enfoque arquitectónico permite desacoplar la captura del análisis, facilitando la reutilización de datos y la extensión futura del sistema.

B.4. Diseño procedimental

El diseño procedimental describe la secuencia de operaciones que ejecuta el sistema en respuesta a las acciones del usuario.

La Figura B.4 muestra el flujo completo del sistema, desde la interacción inicial hasta la generación de resultados.

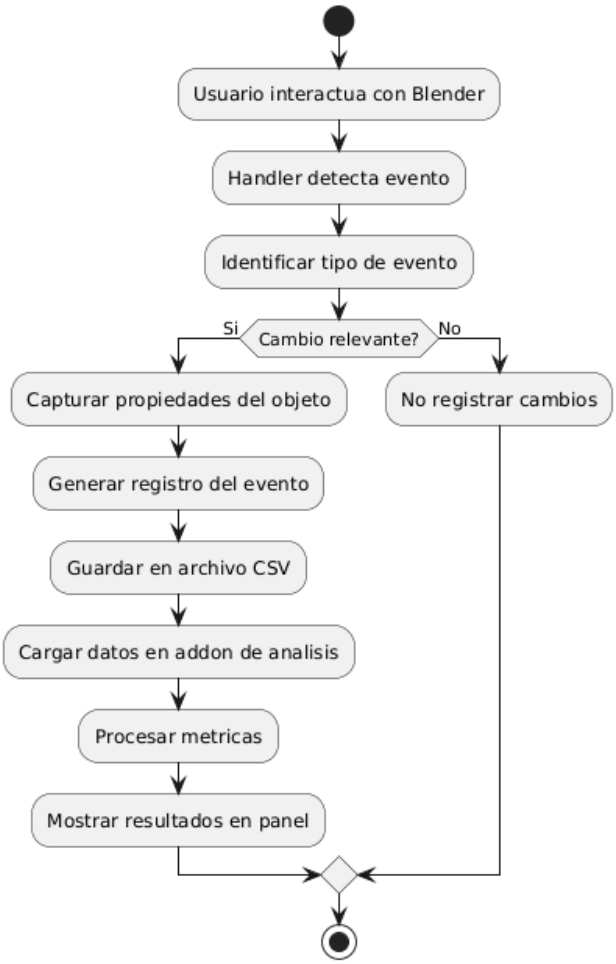

Figura B.4: Flujo procedimental del sistema.

Por su parte, la Figura B.5 describe la interacción temporal entre los distintos componentes del sistema.

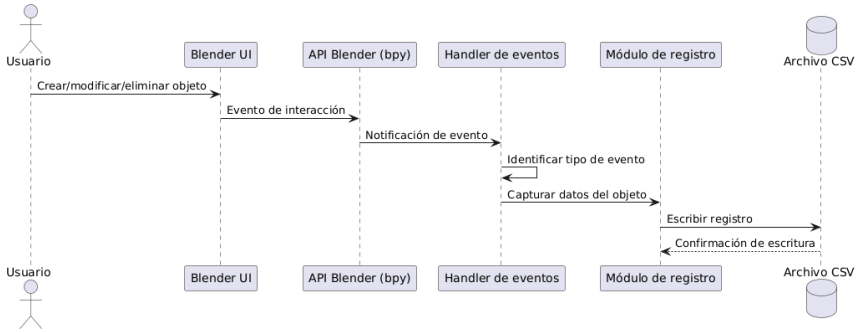

Figura B.5: Diagrama de secuencia del sistema.

El flujo general de ejecución es el siguiente:

1. El usuario realiza una acción en Blender.
2. El sistema detecta el evento mediante los handlers registrados.
3. Se identifican el tipo de acción y el objeto afectado.
4. Se capturan las propiedades relevantes.
5. Se genera un registro estructurado del evento.
6. El registro se almacena en el archivo CSV correspondiente.
7. El addon de análisis procesa los datos y genera métricas.
8. Los resultados se presentan en la interfaz de usuario.

El sistema optimiza el rendimiento evitando el registro de información redundante, almacenando únicamente cambios significativos en el estado de los objetos. Este enfoque orientado a eventos se adapta de forma natural al modelo de ejecución de Blender, permitiendo una integración eficiente con su API.

B.5. Decisiones de diseño relevantes

Durante el desarrollo del sistema se han adoptado varias decisiones clave que condicionan su comportamiento y estructura:

- **Separación en dos addons**: permite desacoplar la captura de datos del análisis, facilitando la mantenibilidad y evolución del sistema.

- **Uso de CSV como formato de persistencia**: proporciona una solución ligera, interoperable y fácilmente depurable.

- **Captura basada en handlers**: permite detectar automáticamente eventos en Blender sin intervención directa del usuario.

- **Procesamiento diferido de métricas**: evita sobrecargar el sistema durante la captura de datos.

- **Registro selectivo de cambios**: reduce el volumen de datos y mejora la eficiencia.

Estas decisiones garantizan un sistema robusto, eficiente y alineado con los requisitos definidos.

Por último, la Figura B.6 presenta una visión global del sistema, integrando los dos addons desarrollados y mostrando el flujo completo de información entre ellos.

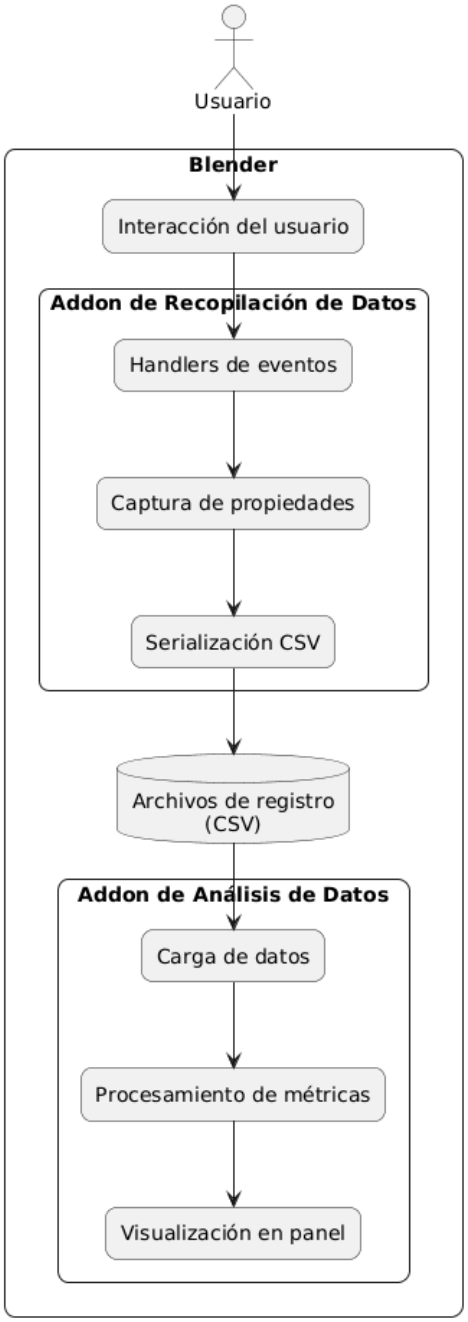

Figura B.6: Arquitectura general del sistema basada en dos addons independientes.

Apéndice C

Documentación técnica de programación

C.1. Introducción

En este apartado se describe la estructura técnica del sistema desarrollado, así como los aspectos necesarios para su instalación, ejecución y mantenimiento. Se detallan los módulos que componen cada uno de los addons, su organización interna y las principales funcionalidades implementadas.

El sistema está formado por dos addons diferenciados: el addon de recopilación de datos, implementado como un script ejecutable dentro de Blender, y el addon de análisis de datos, distribuido como un paquete instalable. Ambos componentes trabajan de forma complementaria mediante el uso de archivos CSV como medio de intercambio de información.

C.2. Estructura de directorios

El proyecto se divide en dos partes principales correspondientes a los dos addons desarrollados.

Addon de recopilación de datos

Este addon se implementa como un único archivo Python:

- **DeteccionDatosV4.py**: script principal encargado de la captura de eventos, detección de cambios en la escena y registro de datos en archivos CSV.

Este enfoque permite una ejecución directa desde el editor de scripting de Blender, facilitando su uso en entornos académicos.

Addon de análisis de datos

Este addon se distribuye como un paquete estructurado en varios módulos:

- **___init___.py**: archivo de inicialización del addon e integración con Blender.
- **metrics_ui.py**: gestión de la interfaz de usuario y panel de visualización de métricas.
- **graphs.py**: generación de gráficos y representaciones visuales.
- **utils.py**: funciones auxiliares para procesamiento de datos.
- **operator.py**: definición de operadores de Blender para la interacción con el sistema.
- **site-packages/**: dependencias externas necesarias para el funcionamiento del addon.

Esta estructura modular facilita la mantenibilidad, escalabilidad y reutilización del código.

C.3. Manual del programador

El sistema ha sido desarrollado siguiendo un enfoque modular y orientado a eventos.

Addon de recopilación

El addon de recopilación se basa en el uso de *handlers* de Blender para detectar eventos en la escena. Su funcionamiento principal consiste en:

- Detectar cambios en la escena mediante eventos de Blender.
- Identificar el tipo de acción realizada (creación, modificación, eliminación).
- Capturar propiedades relevantes del objeto afectado.
- Registrar la información en archivos CSV.

Se han implementado mecanismos para evitar el registro de cambios redundantes, de forma que únicamente se almacenan modificaciones reales en el estado de los objetos.

Además, el sistema incorpora control de ejecución mediante funcionalidades de inicio y parada (Start/Stop), permitiendo al usuario gestionar la captura de datos.

Addon de análisis

El addon de análisis está diseñado de forma modular, separando claramente la lógica de procesamiento de datos de la interfaz de usuario.

Sus componentes principales son:

- **Carga de datos**: lectura de archivos CSV generados por el addon de recopilación.
- **Procesamiento de métricas**: cálculo de estadísticas y métricas a partir de los datos registrados.
- **Visualización**: representación de resultados mediante gráficos y paneles dentro de Blender.

El sistema permite analizar la información de forma flexible, ofreciendo diferentes tipos de visualización y facilitando la interpretación de los datos.

C.4. Compilación, instalación y ejecución del proyecto

Addon de recopilación

El addon de recopilación se ejecuta directamente desde Blender siguiendo estos pasos:

1. Abrir Blender.
2. Acceder al espacio de trabajo *Scripting*.
3. Cargar el archivo `DeteccionDatosV4.py`.
4. Activar la opción *Register*.
5. Ejecutar el script.
6. Guardar el archivo `.blend`.

Una vez ejecutado, el sistema solicita el consentimiento del usuario para la recopilación de datos. A partir de ese momento, la captura de eventos se realiza automáticamente durante el uso del proyecto.

Addon de análisis

El addon de análisis se instala como un addon estándar de Blender:

1. Comprimir el addon en formato `.zip`.
2. Acceder a *Edit* > *Preferences* > *Add-ons*.
3. Seleccionar *Install* e importar el archivo `.zip`.
4. Activar el addon.

Una vez instalado, el usuario puede acceder al panel de análisis y cargar los datos generados para su procesamiento.

C.5. Pruebas del sistema

Se han realizado diferentes pruebas para validar el correcto funcionamiento del sistema.

Pruebas del addon de recopilación

- Registro correcto de creación de objetos.
- Detección de modificaciones en *Edit Mode*.

- Captura de operaciones como duplicado (Shift+D), pegado (Ctrl+V) y eliminación.
- Verificación del funcionamiento de los controles Start/Stop.
- Comprobación de que no se registran cambios redundantes.
- Validación del almacenamiento correcto en archivos CSV.

Pruebas del addon de análisis

- Carga correcta de archivos CSV.
- Verificación del cálculo de métricas.
- Validación de la representación gráfica (escalas, ejes y datos).
- Comprobación de la coherencia de la información mostrada en pantalla.
- Pruebas de funcionamiento de distintos tipos de gráficos.

Estas pruebas permiten garantizar que el sistema funciona de forma consistente, tanto en la captura de datos como en su posterior análisis.

El sistema ha sido validado de forma incremental durante su desarrollo, asegurando la coherencia entre los datos capturados y los resultados obtenidos en el análisis.

Apéndice D

Documentación de usuario

D.1. Introducción

En este apartado se describe el uso del sistema desde el punto de vista del usuario final. Se detallan los requisitos necesarios, el proceso de instalación y las instrucciones de uso de los addons desarrollados.

El sistema está diseñado para ser utilizado principalmente en un entorno educativo, donde un usuario (por ejemplo, un alumno) trabaja con Blender mientras el sistema registra automáticamente su actividad. Posteriormente, estos datos pueden ser analizados mediante un segundo addon destinado a la visualización de métricas.

D.2. Requisitos de usuario

Para utilizar el sistema, el usuario debe disponer de los siguientes elementos:

- Blender instalado (versión compatible con Python 3).
- Conocimientos básicos de uso de Blender.
- Acceso al archivo `.blend` o al script proporcionado.

No se requieren conocimientos de programación para utilizar el sistema.

D.3. Instalación

El sistema consta de dos componentes con procesos de instalación diferentes.

Addon de recopilación de datos

El addon de recopilación se distribuye como un script que debe ejecutarse dentro de Blender:

1. Abrir Blender.
2. Acceder al espacio de trabajo *Scripting*.
3. Cargar el archivo `DeteccionDatosV4.py`.
4. Activar la opción *Register*.
5. Ejecutar el script.
6. Guardar el archivo `.blend`.

Una vez realizado este proceso, el sistema queda integrado en el archivo de trabajo.

Addon de análisis de datos

El addon de análisis se instala como un addon estándar de Blender:

1. Acceder a *Edit* > *Preferences* > *Add-ons*.
2. Seleccionar la opción *Install*.
3. Importar el archivo comprimido del addon (`.zip`).
4. Activar el addon.

D.4. Manual del usuario

Uso del addon de recopilación

Una vez ejecutado el script, el sistema solicita al usuario su consentimiento para la recopilación de datos. Tras aceptar, el addon comienza a registrar automáticamente la actividad realizada en Blender.

El usuario puede trabajar de forma habitual, realizando operaciones como:

- Crear objetos.
- Modificar geometría.
- Cambiar de modo de edición.
- Eliminar elementos.

El sistema registra estas acciones de forma transparente, sin requerir intervención adicional.

Durante el uso, el sistema almacena la información en archivos CSV asociados al proyecto. Estos datos se generan automáticamente al guardar el archivo `.blend`.

El usuario no necesita realizar ninguna acción adicional para registrar su actividad.

Uso del addon de análisis

El addon de análisis permite visualizar los datos registrados previamente.

Para utilizarlo:

1. Abrir Blender con el addon instalado.
2. Acceder al panel de análisis.
3. Cargar los archivos CSV generados.
4. Seleccionar las métricas o gráficos a visualizar.

El sistema mostrará información sobre la actividad realizada, incluyendo métricas y representaciones visuales.

Flujo de uso completo

El flujo típico de uso del sistema es el siguiente:

1. El usuario abre Blender y ejecuta el addon de recopilación.
2. Acepta el consentimiento de recopilación de datos.
3. Guarda el archivo `.blend`, generando los datos.
4. Realiza su trabajo de modelado de forma habitual.
5. Entrega el archivo o los datos generados.
6. El analista o profesor utiliza el addon de análisis para visualizar métricas.

Este proceso permite registrar la actividad del usuario de forma automática y analizar posteriormente su comportamiento durante el modelado.

Apéndice E

Anexo de sostenibilización curricular

E.1. Introducción

En este anexo se presenta una reflexión sobre los aspectos de sostenibilidad relacionados con el desarrollo del Trabajo de Fin de Grado. En particular, se analizan las implicaciones del proyecto desde el punto de vista tecnológico, educativo y social, en línea con las competencias de sostenibilidad promovidas por la CRUE.

El sistema desarrollado, basado en la recopilación y análisis de datos durante el uso de Blender, permite abordar la sostenibilidad desde una perspectiva digital, centrada en la eficiencia, la optimización de procesos y la mejora del aprendizaje.

E.2. Sostenibilidad tecnológica

Desde el punto de vista tecnológico, el proyecto promueve la eficiencia en el uso de recursos. El sistema ha sido diseñado para registrar únicamente cambios relevantes en la interacción del usuario, evitando la generación de datos redundantes. Esta decisión reduce el volumen de información almacenada y, por tanto, el consumo de recursos de almacenamiento y procesamiento.

Además, el uso de archivos CSV como formato de persistencia contribuye a la simplicidad y ligereza del sistema, evitando dependencias complejas

o infraestructuras costosas. Este enfoque permite minimizar el impacto energético asociado al procesamiento de datos.

Asimismo, el desarrollo se basa en herramientas de software libre como Blender y Python, lo que favorece la sostenibilidad tecnológica al promover soluciones accesibles, reutilizables y mantenibles en el tiempo.

E.3. Sostenibilidad educativa

El proyecto tiene una clara aplicación en el ámbito educativo, donde permite analizar el proceso de aprendizaje de los usuarios, más allá del resultado final. Mediante la recopilación de datos de interacción, es posible estudiar cómo los estudiantes trabajan en entornos de modelado 3D, identificar patrones de comportamiento y detectar posibles dificultades.

Este enfoque contribuye a una educación más sostenible, ya que permite mejorar los procesos de enseñanza y aprendizaje, adaptándolos a las necesidades reales de los estudiantes. En lugar de centrarse únicamente en la evaluación del resultado final, el sistema proporciona herramientas para comprender el proceso, lo que favorece un aprendizaje más eficiente y personalizado.

E.4. Sostenibilidad social

Desde el punto de vista social, el uso de software libre garantiza la accesibilidad del sistema a un amplio número de usuarios, sin barreras económicas. Blender y Python son herramientas ampliamente disponibles, lo que permite que el sistema desarrollado pueda ser utilizado en diferentes contextos educativos y profesionales.

Además, el sistema incorpora mecanismos de consentimiento para la recopilación de datos, respetando la privacidad del usuario y cumpliendo con la normativa vigente. Este aspecto refuerza el compromiso ético del proyecto, garantizando un uso responsable de la información.

E.5. Conclusión

En conjunto, el proyecto contribuye a la sostenibilidad desde múltiples perspectivas. Por un lado, promueve la eficiencia tecnológica mediante la optimización del registro y procesamiento de datos. Por otro, aporta valor

en el ámbito educativo al mejorar la comprensión del proceso de aprendizaje. Finalmente, garantiza la accesibilidad y el respeto a la privacidad, alineándose con principios de sostenibilidad social.

Estas características reflejan la integración de competencias de sostenibilidad en el desarrollo del Trabajo de Fin de Grado, demostrando una visión global del impacto de la tecnología en la sociedad.